Scalable Analysis of Vegetation Anomalies Preceding Plant Disease Outbreaks

Mountain Pine Beetle Outbreak Detection | Boulder County, CO

Group – 02: Lake it Easy + Pulse Crew
Thrisha Ambareesharaje Urs Urs
Sohum Damani
Vedant Borkute
Shreyangshu Bera
Yashaswini Diggavi

Why Does This Matter at Scale?

Background and Motivation

Forest diseases such as the Mountain Pine Beetle have caused widespread ecological damage across large regions, particularly in remote forests where continuous ground monitoring is not feasible. Because visible symptoms appear only after months of spread, interventions often come too late to prevent large-scale loss. This creates a strong need for early detection.

Satellite time-series data enables continuous observation of vegetation health and captures subtle spectral changes, such as shifts in NDVI and NDMI, before they are visible to the human eye. Leveraging these early signals allows for proactive detection of stress patterns, helping mitigate ecological damage while demonstrating the importance of data-driven environmental monitoring.

Project Overview & Big Data Relevance

This project develops a scalable pipeline to detect vegetation anomalies using multi-year satellite imagery from Landsat and Sentinel-2. The system processes large volumes of raster data by applying forest masking, computing vegetation indices, and identifying anomalies through Z-score–based statistical analysis. Designed as a unified pipeline, it operates consistently across datasets with varying resolutions, time spans, and data sizes. The problem inherently reflects key Big Data characteristics, including high data volume, multi-spectral and spatiotemporal variety, and the need for scalable processing. By integrating cloud-native storage (COGs on S3), spatial indexing (PostGIS), and distributed computation (PySpark), the system enables efficient large-scale environmental analysis.

Datasets — Three Scales, One Pipeline

We analyzed three real satellite datasets separately through the same scalable pipeline.

Dataset A

Period	2015 – 2026
Resolution	50 m resolution
Raw Data	4.3 GB
Objects	594 objects
Frequency	~5 images / month

Longest temporal baseline for historical anomaly detection

Dataset B

Period	2024 – 2026
Resolution	30 m resolution
Raw Data	2.8 GB
Objects	96 objects
Frequency	Recent high-res data

Most recent imagery for current outbreak monitoring

Dataset C

Period	2019 – 2026
Resolution	20 m resolution
Raw Data	10.7 GB
Objects	109 images
Frequency	Highest spatial detail

Finest resolution for detailed stress mapping

Why not 100 GB? We selected only required spectral bands (Red, NIR, SWIR1) and applied a cloud cover cap during GEE export, reducing raw file size. All three datasets run through the identical COG + PostGIS + PySpark pipeline, validating scalability across different volumes, resolutions, and time ranges.

Existing Work & Our Position

Vegetation Time-Series Analysis

- BFAST (Verbesselt et al. 2010) — seasonal breakpoints
- LandTrendr (Kennedy 2010) — Landsat segmentation
- Xie et al. 2025 — near-real-time anomaly detection

Our approach: We use simpler Z-score method that scales better in Spark

Forest Disturbance & MPB Detection

- Meddens et al. 2012 — MPB spectral trajectories
- Wulder et al. 2006 — remote sensing survey
- Coops et al. 2009 — vigor estimates

Our approach: We target green-attack phase: earliest detectable stress

Geospatial Big Data Systems

- SpatialHadoop (Eldawy 2015) — spatial MapReduce
- Apache Sedona — spatial Spark
- GEE (Gorelick 2017) — planetary analysis

Our approach: We use PySpark + PostGIS rather than full in-Spark spatial joins

Cloud-Native Raster Infrastructure

- Cloud Optimized GeoTIFF (COG) standard
- STAC (Radiant Earth) — catalog spec
- Planetary Computer (Microsoft 2021)

Our approach: We store all 15-20 GB as COGs on S3 with rasterio streaming

End-to-End Pipeline Architecture

Data Ingest
GEE + S3
COG ETL

Spatial Index
PostGIS
GIST+B-tree

Forest Mask
ESA WorldCover
rasterio

Veg. Indices
PySpark
NDVI/NDMI

Anomaly Detect
Z-score
baseline

Output + Viz
Parquet
Timelapses

Input Data

- 3 datasets: 50m / 30m / 20m
- Periods: 2015-26, 2024-26, 2019-26
- 10.2 GB, 6.2 GB, 14.8 GB (COG)

Processing Engine

- Apache PySpark 3.5
- Rasterio (S3 streaming)
- GDAL COG conversion

Storage Stack

- AWS S3 (COG objects)
- PostgreSQL + PostGIS
- Parquet result files

Key Algorithms

- NDVI/NDMI computation
- Monthly temporal composites
- Z-score anomaly detection

20m Dataset — Collection & Tiling (Sentinel-2 SR Harmonized)

Bands: B4 (Red) • B8 (NIR) • B11 (SWIR1) | Cloud cap: 40% | Up to 10 images/month | Scale: 20m

Collection Pipeline

1. Filter in GEE
2. Split into 3×4 Tile Grid (12 tiles)
3. Validate & Merge Tiles
4. Upload to S3
5. COG Convert & Re-index

3×4 Tile Grid & Key Challenges

T1	T2	T3	T4
T5	T6	T7	T8
T9	T10	T11	T12

4 cols × 3 rows = 12 tiles per scene

⚠ **Tile validity gate:** Need at least 8 tiles to be passed such that each are more than 50Kb

⚠ **Merge nodata alignment:** rasterio.merge uses nodata=-9999. Without this, masked cloud pixels default to 0, corrupting NDVI values at boundaries.

⚠ **10 MB size floor:** Minimum merged file should be above 10 Mb

AWS Infrastructure — COG Conversion & Spatial Indexing

Why these choices? How they work together.

AWS Infrastructure

S3 Bucket
vegetation-anomaly-cogs
Separate prefixes per dataset:
- 20m_resolution
- 30m_resolution
- 50m_resolution

RDS PostgreSQL

PostGIS extension enabled.
Stores metadata only — not raster pixels.
Keeps DB lightweight.

Why Cloud Optimized GeoTIFF?

Plain TIFF

- Full file must download before any pixel can be read
- No internal structure for partial access
- ~100 MB transfer just to read one tile window

COG (our choice)

- Internally tiled at 512×512 px blocks
- HTTP byte-range requests: only request the block(s) you need
- Overview levels [2,4,8,16] for multi-scale fast access
- LZW compression: 3–5× smaller on S3

Why These Index Types?

GIST Index *on geom (Polygon)*

→ 12–18 ms vs 280–450 ms without index (15–25× faster)

B-tree Index *on acquisition_date*

→ Enables sub-millisecond date range filtering

Composite Index on (tile_id, date)

→ Avoids duplicate ingestion overhead

Cloud-Native Storage + Spatiotemporal Index

Cloud Optimized GeoTIFF (COG)

Internally tiled: 512×512 px tiles for sub-file HTTP reads

LZW compressed: Reduces S3 storage ~3-5x

Overview levels: [2,4,8,16] for fast zoom-out reads

Partial reads: Only requested tiles transferred from S3

PostGIS Spatiotemporal Index

GIST Spatial Index: ST_Intersects bounding box queries

B-tree Date Index: Fast acquisition_date BETWEEN filters

Composite Index: (tile_id, date) for scene lookups

ST_MakeEnvelope: Stores WGS84 bbox for each COG

From Raw Pixels to Monthly Vegetation Indices

1

Data Fetching and Validation

Concurrently/Serially/Query-based fetch and stream spectral bands (Red, NIR, SWIR1) directly from S3 COGs via *AWSSession*. Then, check for data integrity and perform standardization.

2

Forest Masking

Reprojects WorldCover (Class 10 <-> forest) data onto the HLS grid using *rasterio*, reducing downstream processing volume by 30–70% by discarding non-forest pixels, essentially acting as a filter

3

Vegetation & Moisture Indices calculation

Calculate NDVI (an indication of 'greenness') and NDMI (an indication of moisture content) using efficient vectorized expressions, execution on Spark action.

4

Data Harmonization

Aggregate NDVI and NDMI per pixel per month, to harmonize the possibly irregular data using Spark groupBy(). Deliberate choice to not interpolate the data in order to ensure only actual-data backed pixel-month anomalies are flagged.

Pipeline Runs — Three Scene Fetch Modes Compared

Same code, same data but different scene discovery strategies.

▶ Serial Mode

How it discovers scenes:

list_cogs_from_s3() scans 20m_resolution/cog/ prefix on S3. One scene at a time, no parallelism. Blocks on each rasterio.open() call.

```
Sequential order:
veg_2021-05-06
veg_2021-06-05
veg_2021-06-10
veg_2021-09-18
veg_2021-12-22 ...
```

Pros:

+ Simplest to debug
+ Guaranteed order
+ Low memory pressure

Cons:

– Slowest for large archives
– One S3 read at a time
– No I/O overlap

⚡ Concurrent Mode

How it discovers scenes:

list_cogs_from_s3() same as serial. But reads with ThreadPoolExecutor(max_workers=24) — up to 24 S3 COG reads in parallel via rasterio AWSSession.

```
Non-sequential (proves parallel):
veg_2021-05-06
veg_2021-01-11  ← out of order
veg_2022-02-20  ← jumps ahead
veg_2021-12-22 ...
```

Pros:

+ Up to 24× faster scene reads
+ Saturates S3 bandwidth
+ Same results as serial

Cons:

– All COGs in folder loaded (no date filter)
– Memory usage scales with workers
– Order not guaranteed

🔍 Query Mode

How it discovers scenes:

run_spatiotemporal_query() hits PostGIS with dataset='20m_sentinel2' AND date BETWEEN '2019-01-01' AND '2022-12-31'. Returns only relevant S3 URLs. Then concurrent reads.

```
DB returns 74 scenes:
(only 2019-01-01 to
2022-12-31 window)
not all 109 in folder
0.37s query latency
```

Pros:

+ Fastest discovery: 0.37s
+ 32% fewer scenes (74 vs 109)
+ Date + dataset filtered
+ Production-grade, reproducible

Cons:

– Requires populated DB
– Query window must match baseline needs

Fetching Metrics — Side-by-Side Results from All Three Runs

With Identical anomaly detection results, we obtain different efficiency metrics for the three methods

Metric	Serial	Concurrent	Query (DB)
Scene source	S3 prefix list	S3 prefix list	PostGIS index
Scenes fetched	109	109	**74 (32% fewer)**
Latency	0.65	0.64	**0.37s**
Scenes Fetched per sec	168.38	171.2	**198.43**
Scene read method	Sequential loop	24 parallel threads	24 parallel threads
Data window	ALL in folder	ALL in folder	**2019-01 → 2022-12**
Baseline months	All pre-eval	All pre-eval	**40 months**
Parallelism in Spark	48	48	2

Fetching Results — Side by Side from all three runs, but with a complete data timeframe

Here, we fetch the complete data (109 scenes) with all three methods, but obtain a surprising result, a worsened result with the Query method.

Metric	Serial	Concurrent	Query
Scene source	S3 prefix list	S3 prefix list	PostGIS index
Scenes fetched	109	109	**109**
Latency	0.68	0.69	**1.64s**
Scenes Fetched per sec	161.19	159.06	**66.32**
Scene read method	Sequential loop	24 parallel threads	24 parallel threads
Data window	ALL in folder	ALL in folder	ALL in folder
Task Executor ms	5325	5128	**6664**
Parallelism in Spark	48	48	48

Data Transformation & Anomaly Detection Pipeline Products

Analytical Products (parquet)

anomaly_events
Dataset identifying a pixel-month which deviated from baseline (*anomaly*)

plot_stats
Time-series history of every pixel and indices for further granular research.

monthly_stats
Aggregated table with monthly averages of vegetation indices, detected anomalies

pixel_coords
A spatial lookup table that maps Pixel IDs back to physical Latitude/Longitude coordinate/

Visual Products (gif's)

NDVI heatmap of forested region over evaluation period

(green=healthy, red=stressed)

NDMI heatmap of forested region over time

(blue=moist, yellow=dry)

A false-color composite from the Red, NIR, SWIR bands.
(vegetation=magenta, bare soil=greenish)

Note: These are timelapses for selected evaluation period, and consist of forested pixels.

Metadata (csv's)

params

pipeline configuration corresponding to the run (thresholds, scene fetch method, evaluation date ranges, scene resolution)

pipeline_metrics

Performance metrics for every stage of the run e.g.
Spark throughput (MB/s), processing latency etc.

Z-Score Statistical Anomaly Detection

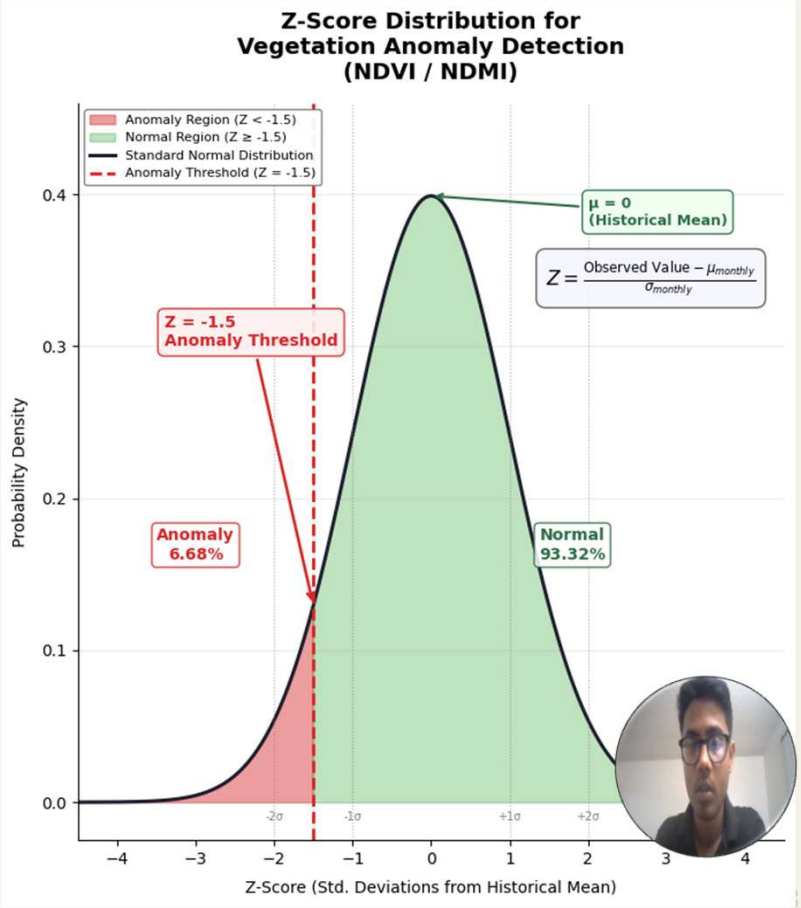
How It Works

1. **Build Baseline:** For every pixel & calendar month: compute mean (μ) and std (σ) from all years BEFORE the evaluation window

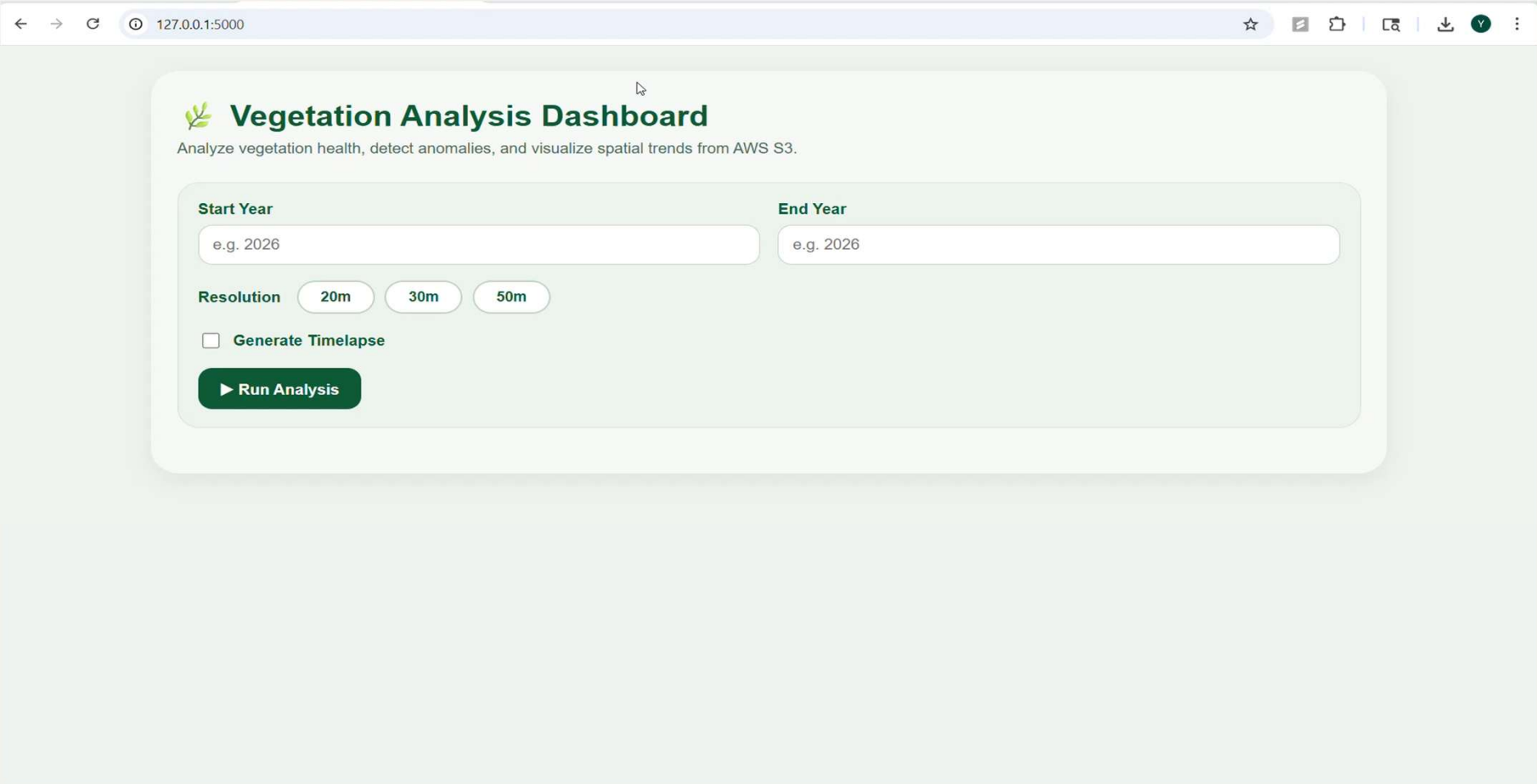
2. **Compute Z-Score:** Z = (Observed NDVI − μ_month) / σ_month
Compares each observation to its pixel-specific seasonal norm

3. **Flag Anomalies:** Z < −1.5 threshold → flagged as anomaly
Lower threshold increases recall for early stress detection

4. **Export Results:** Parquet files to S3 with Z-scores, flags, and baseline stats. Also raster maps and timelapse GIFs.



Live Demo — Vegetation Anomaly Detection Dashboard:



Performance Across Resolutions:

Conclusion

GIST spatial indexing delivers 15-25× query speedup over sequential scans — critical for interactive spatiotemporal analysis at Big Data scale.

PySpark distributes pixel-level vegetation index computation effectively; throughput scales approximately linearly with data volume (3 → 10 years).

The COG + S3 + PostGIS + PySpark stack validates a practical, cloud-native architecture for large-scale raster time-series analysis in ecology and beyond.